

Move Constructor and Move Assignment

Absolutely. This is the **next step after Rule of Three** and is very important for C++ interviews.

1. Why do we need Move Semantics?

Suppose we have:

```
class Student {
    int* marks;

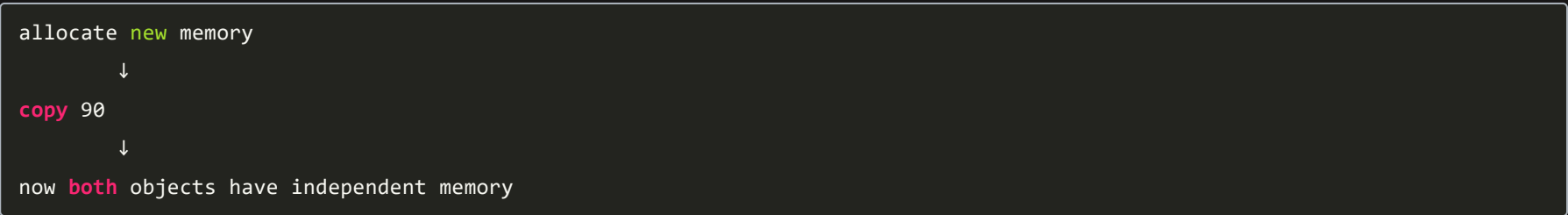
public:
    Student(int x) {
        marks = new int(x);
    }

    ~Student() {
        delete marks;
    }
};
```

Now:

```
Student s1(90);
Student s2 = s1;
```

Copying requires:



That's necessary when we need **two independent objects**.

But what if `s1` is a temporary object that we're never going to use again?

```
Student s2 = Student(90);
```

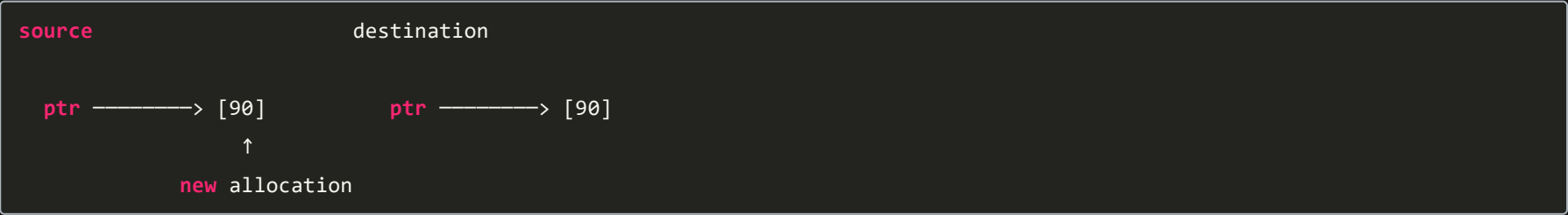
Why allocate another memory block and copy `90`?

We could simply **steal the resource** from the temporary object.

That's what **move semantics** does.

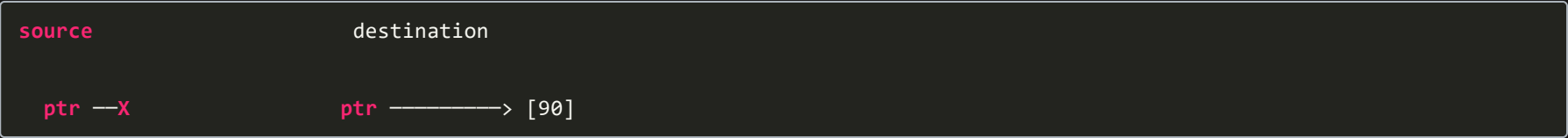
2. Copy vs Move

Copy



The destination gets its **own resource**.

Move



The destination **takes ownership** of the existing resource.

The source is left in a **valid but unspecified state**.

For our pointer example, we'll set it to `nullptr`.

3. Move Constructor

A move constructor looks like:

```
ClassName(ClassName&& other);
```

Notice:

```
&&
```

This is an **rvalue reference**.

Example:

```
class Student {
    int* marks;

public:
    Student(int x) {
        marks = new int(x);
    }

    // Move constructor
    Student(Student&& other) {
        marks = other.marks;
        other.marks = nullptr;
    }

    ~Student() {
        delete marks;
    }
};
```

Now:

```
Student s1(90);

Student s2 = std::move(s1);
```

Conceptually:

Before:

s1 —————> [90]
s2 —————> ?

After:

s1 —> nullptr
s2 —————> [90]

No new memory was allocated.

We simply transferred ownership.

4. Why set `other.marks = nullptr`?

This is **extremely important**.

Suppose:

```
Student(Student&& other) {
    marks = other.marks;
}
```

After moving:

s1 ————┐
 └─> [90]
s2 ————┐

Both objects point to the same memory.

Then destructors run:

```
s2 -> delete marks
s1 -> delete marks again
```

✱ **Double delete / undefined behavior.**

So:

```
marks = other.marks;
other.marks = nullptr;
```

transfers ownership safely.

5. Move Assignment Operator

Move constructor handles **creating a new object**:

```
Student s2 = std::move(s1);
```

But what about an already-existing object?

```
Student s1(90);
Student s2(80);

s2 = std::move(s1);
```

Here `s2` already exists.

We need a **move assignment operator**:

```
Student& operator=(Student&& other);
```

Implementation:

```
Student& operator=(Student&& other) {
    if (this != &other) {
        delete marks;          // release current resource

        marks = other.marks;    // steal resource

        other.marks = nullptr;  // leave source safe
    }

    return *this;
}
```

Now:

```
Before:

s1 ———> [90]
s2 ———> [80]

s2 = std::move(s1);

After:

s1 ———> nullptr
s2 ———> [90]
```

The old `[80]` is deleted.

6. Complete Rule of Five Example

This is a very good interview example.

```

class Student {
    int* marks;

public:
    // Constructor
    Student(int x) {
        marks = new int(x);
    }

    // 1. Copy constructor
    Student(const Student& other) {
        marks = new int(*other.marks);
    }

    // 2. Copy assignment
    Student& operator=(const Student& other) {
        if (this != &other) {
            delete marks;
            marks = new int(*other.marks);
        }
        return *this;
    }

    // 3. Move constructor
    Student(Student&& other) noexcept {
        marks = other.marks;
        other.marks = nullptr;
    }

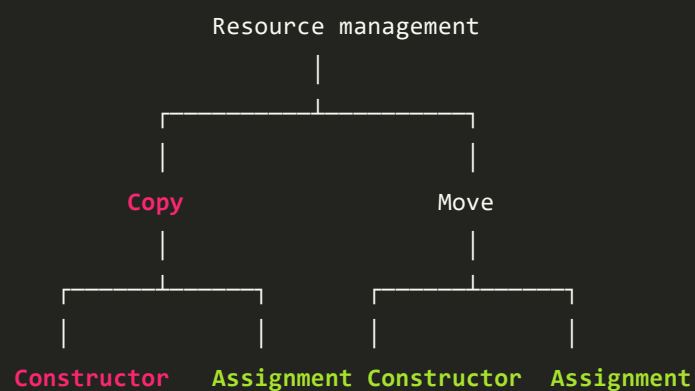
    // 4. Move assignment
    Student& operator=(Student&& other) noexcept {
        if (this != &other) {
            delete marks;

            marks = other.marks;
            other.marks = nullptr;
        }
        return *this;
    }

    // 5. Destructor
    ~Student() {
        delete marks;
    }
};

```

That's the **Rule of Five**.



7. What exactly is `std::move`?

This is a **very common** interview trap.

Many beginners think:

"`std::move` moves the object."

Not exactly.

`std::move` **does not actually move anything**.

It essentially **casts an expression into an rvalue**, allowing move operations to be selected.

Conceptually:

```
std::move(s1)
```

says:

```
"Treat s1 as an object whose resources can be transferred."
```

Then the appropriate move constructor/assignment can steal those resources.

8. Example of `std::move`

```
Student s1(90);

Student s2 = std::move(s1);
```

Without `std::move`:

```
Student s2 = s1;
```

`s1` is an **lvalue**, so the copy constructor is selected:

```
Student(const Student& other)
```

With:

```
Student s2 = std::move(s1);
```

`s1` is treated as an **rvalue**, so the move constructor can be selected:

```
Student(Student&& other)
```

9. lvalue vs rvalue

You need this distinction for move semantics.

lvalue

Has an identifiable location/name.

```
Student s1(90);
```

`s1` is an lvalue.

rvalue

Usually a temporary/value that doesn't represent a persistent object you intend to keep using.

```
Student(90)
```

is an rvalue.

So:

```
Student s2 = Student(90);
```

can use move semantics, although **copy elision may eliminate the move entirely**.

10. Very Important: Moved-from Object

After:

```
Student s2 = std::move(s1);
```

you should **not assume** `s1` still contains its original resource.

For our implementation:

```
s1.marks == nullptr
```

But the general C++ rule is:

```
A moved-from object remains valid but its state is unspecified, unless the type specifies more.
```

You can safely:

```
s1 = Student(50);
```

or destroy it.

But don't assume it still has its old value.

11. Why `noexcept` on Move Constructor?

You'll frequently see:

```
Student(Student&& other) noexcept
```

Why?

Because standard containers such as `std::vector` prefer using a move constructor during reallocation **when it is guaranteed not to throw**.

For example:

```
std::vector<Student> students;
```

When the vector grows, it may need to move existing elements to new storage.

If moving is `noexcept`, the vector can safely use the move operation.

Interview answer:

We typically mark move constructors and move assignment operators `noexcept` because standard containers can use them more efficiently during operations such as vector reallocation, where throwing during a move could complicate exception safety.

12. `std::move` doesn't guarantee a move

Another **excellent interview trap**.

```
std::move(x)
```

doesn't guarantee that a move constructor will be called.

It only converts `x` into an rvalue expression.

If the class doesn't have a move constructor, the compiler may fall back to copying.

For example:

```
class A {
public:
    A(const A&);
};
```

Then:

```
A a;
A b = std::move(a);
```

can still invoke:

```
A(const A&)
```

because an rvalue can bind to a `const` lvalue reference.

13. The most important distinction

Memorize this:

Operation	Purpose	Copy constructor	Create object by copying resources	Copy assignment	Existing object copies resources	Move constructor	Create object by stealing resources	Move assignment	Existing object steals resources	<code>std::move</code>	Cast expression to an rvalue	<code>&&</code>	Rvalue reference	<code>noexcept</code>	Move is guaranteed not to throw
-----------	---------	------------------	---	-----------------	---	------------------	--	-----------------	---	------------------------	-------------------------------------	-------------------------	------------------	-----------------------	---------------------------------

Interview Questions You Should Be Able to Answer

Q1. Why do we need move semantics?

Because copying dynamically allocated resources can be expensive. When the source is a temporary or no longer needed, we can transfer ownership instead of allocating and copying.

Q2. What's the difference between copy and move?

Copy **duplicates the resource**. Move **transfers ownership of the resource**.

Q3. Does `std::move` actually move anything?

No. It casts an expression to an rvalue, enabling move constructors/assignment operators to be selected.

Q4. Why do we set the source pointer to `nullptr`?

To ensure the moved-from object doesn't delete a resource that now belongs to the destination.

Q5. What is Rule of Five?

If a class manually manages a resource, it may need:

```
Destructor
Copy constructor
Copy assignment
Move constructor
Move assignment
```

Q6. Why is move constructor usually `noexcept`?

So standard containers can safely prefer moving over copying during operations such as reallocation.

 The mental model I'd use in an interview

Think of an object as **owning a resource**:

```
COPY
———

"I want TWO owners with independent resources."

A —> Resource 1
B —> Resource 2


MOVE
———

"I don't need A's resource anymore.
B can TAKE IT."

A —> nullptr
B —> Resource
```

And connect it directly to what you just learned:

```
Rule of 3
  ↓
Destructor
Copy Constructor
Copy Assignment
  ↓
Resource management
  ↓
Rule of 5
  +
Move Constructor
Move Assignment
```

One sentence to remember for interviews:

Copy duplicates ownership; move transfers ownership, and `std::move` merely enables that transfer by treating an object as an rvalue.